Universidade Federal de Minas Gerais
Instituto de Ciências Exatas
Departamento de Ciência da Computação

Aprendizado por Reforço 2026/2
Professor: Luiz Chaimowicz
Monitor: Daniel Terra Gomes

Trabalho Prático: Deep Q-Network no Lunar Lander

Valor: 15 pontos
Data de entrega: 27 Novembro de 2026

1 Introdução

O Deep Q-Network (DQN) representa um marco no avanço do aprendizado por reforço, ao combinar Q-Learning com redes neurais profundas para lidar com espaços de estados de alta dimensionalidade. Sua tarefa neste trabalho consiste em implementar e validar o DQN no ambiente Lunar Lander, disponibilizado pela biblioteca Gymnasium. Além disso, você deverá conduzir experimentos para avaliar a eficácia da solução proposta, incluindo uma análise do impacto de diferentes hiperparâmetros e a comparação de desempenho com uma implementação de referência, fornecida pela biblioteca Stable Baselines3.

2 Lunar Lander

O ambiente Lunar Lander, da biblioteca Gymnasium, simula a tarefa de controlar uma nave em queda para realizar um pouso suave e seguro na superfície da Lua. O cenário consiste em uma área com uma zona de pouso central, e o agente deve aprender a acionar os motores da nave de forma eficiente para estabilizá-la, desacelerar sua queda e tocar o solo com segurança dentro da área demarcada.

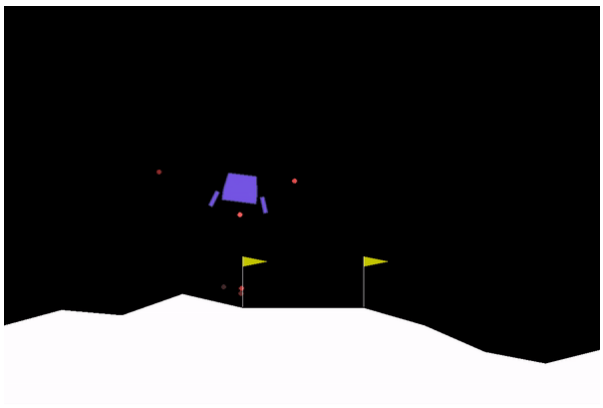


Figura 1: Ilustração do Ambiente Lunar Lander

O estado observado pelo agente a cada passo é representado por um vetor contínuo de oito dimensões. Esse vetor inclui a posição horizontal e vertical da nave, suas velocidades lineares nas direções x e y, o ângulo da nave, a velocidade angular, e dois valores booleanos que indicam se a perna esquerda ou a perna direita estão em contato com o solo. A nave sempre inicia um

episódio em uma posição elevada, no centro do ambiente, e com uma força aleatória aplicada a seu centro de massa.

As ações possíveis pertencem a um espaço discreto com quatro opções: acionar o motor principal, que gera impulso vertical em relação à orientação da nave; acionar o motor lateral esquerdo, que aplica torque e força horizontal; acionar o motor lateral direito, com efeito oposto; ou não realizar nenhuma ação. O agente deve aprender a combinar essas ações para controlar a trajetória e a orientação da nave durante a descida.

A função de recompensa do ambiente foi projetada para incentivar um pouso estável e penalizar comportamentos perigosos ou ineficientes. O agente recebe recompensas proporcionais à sua velocidade, à inclinação da nave e à distância em relação ao centro da zona de pouso. Há também penalizações pelo uso de combustível, refletidas no acionamento dos motores. Por outro lado, o agente recebe recompensas positivas cada vez que uma das pernas toca o solo e, ao final do episódio, uma grande recompensa por um pouso bem-sucedido ou uma grande penalização em caso de colisão ou se a nave sair da área de simulação.

O episódio termina quando a nave pousa corretamente, colide com o solo ou sai dos limites da área visível. O objetivo do agente é aprender uma política que maximize a recompensa acumulada ao longo do tempo, realizando pousos seguros e eficientes, minimizando o consumo de combustível e evitando colisões. Para mais detalhes sobre o Lunar Lander, consulte a documentação disponivel em: https://gymnasium.farama.org/environments/box2d/lunar_lander/

3 Tarefas do Trabalho

3.1 Implementar um Deep Q-Network

Você deverá implementar o DQN para solucionar o ambiente Lunar Lander utilizando apenas bibliotecas de manipulação numérica e aprendizado profundo, como NumPy e PyTorch (ou TensorFlow). Nesta etapa, não é permitido o uso de bibliotecas que oferecem implementações prontas do DQN ou de seus componentes principais (exceto pelos módulos de redes neurais profundas). Exemplos de bibliotecas **não** permitidas incluem, mas não se limitam a: Stable Baselines, Keras-RL, RLlib, entre outras.

Sua implementação deve incluir todos os elementos essenciais do DQN: a rede neural para estimativa dos Q-values, a rede alvo e seu mecanismo de atualização, o buffer de replay, a política de exploração ε-greedy com decaimento, além dos processo de treinamento e avaliação do agente.

3.2 Realizar a Busca de Hiperparâmetros

Após a implementação do DQN, você deverá realizar experimentos para buscar uma combinação eficaz de hiperparâmetros que maximize o desempenho do agente. Você deve testar diferentes valores e registrar os resultados. Entre os hiperparâmetros que devem ser explorados estão:

- Taxa de aprendizado;
- Fator de desconto;
- Tamanho do replay buffer;

- Tamanho do batch;
- Frequência de atualização da rede alvo;
- Parâmetros da política e-greedy (valor inicial de epsilon, taxa de decaimento, valor mínimo);
- Número de passos (ou episódios) de treinamento.

3.3 Comparação com o Stable Baselines3

Após finalizar a busca e o ajuste dos hiperparâmetros do seu agente DQN, compare o desempenho da sua implementação com a versão do DQN disponibilizada pela biblioteca Stable Baselines3, utilizando a configuração padrão. Para isso, consulte a documentação oficial do Stable Baselines3 (`https://stable-baselines3.readthedocs.io`) para instruções sobre instalação e execução do algoritmo.

3.4 Documentação

Produza uma documentação clara de seu trabalho, com os seguintes elementos:

Capa: Indique o título do trabalho, seguido do nome completo e do número de matrícula dos(as) autores(as). Não é necessário ter uma página exclusiva para a capa.

Introdução: Contextualize seu trabalho. Inclua uma breve explicação do problema abordado e um resumo de sua solução.

Implementação: Detalhe a solução desenvolvida, as tecnologias e bibliotecas utilizadas, bem como as principais decisões de implementação.

Avaliação Experimental: Apresente os experimentos realizados para validar a eficácia da solução. Descreva os processos de treinamento e teste do agente, destacando seu desempenho ao longo do tempo e os resultados obtidos. Explique a metodologia adotada para a busca de hiperparâmetros, comparando o desempenho de diferentes configurações. Inclua uma análise comparativa entre o agente implementado e a solução de referência fornecida pelo Stable Baselines 3, discutindo diferenças e semelhanças de desempenho e possíveis causas para esses resultados.

Conclusão: Resuma os principais resultados alcançados. Aponte possíveis melhorias e direções futuras para a sua solução.

Trabalhos sem documentação receberão nota zero.

4 Entrega

Deverá ser submetido um arquivo *.zip* contendo (i) a documentação do trabalho, (ii) o código-fonte completo da implementação, (iii) um arquivo README com instruções sobre como executar sua implementação e reproduzir os resultados detalhados em sua documentação. O README deve incluir a lista de pacotes e dependências necessários para o funcionamento do código.